# LedgerPilot

# Procurement, reconciled.

Source evidence behind every exception.
A human decision before every export.

| SYNTHETIC BATCH | READY FOR REVIEW | HELD FOR CLARIFICATION |
| --- | --- | --- |
| **$11,050** | **$4,780** | **$6,270** |

8 invoices | 5 purchase orders | 5 receipts | No real transactions

# A plausible invoice can still be wrong.

Start with the source documents, then decide what needs clarification.

### Purchase order

PO-2402

**100 units ordered**

### Goods receipt

GR-402

**80 units received**

### Invoice

INV-1002

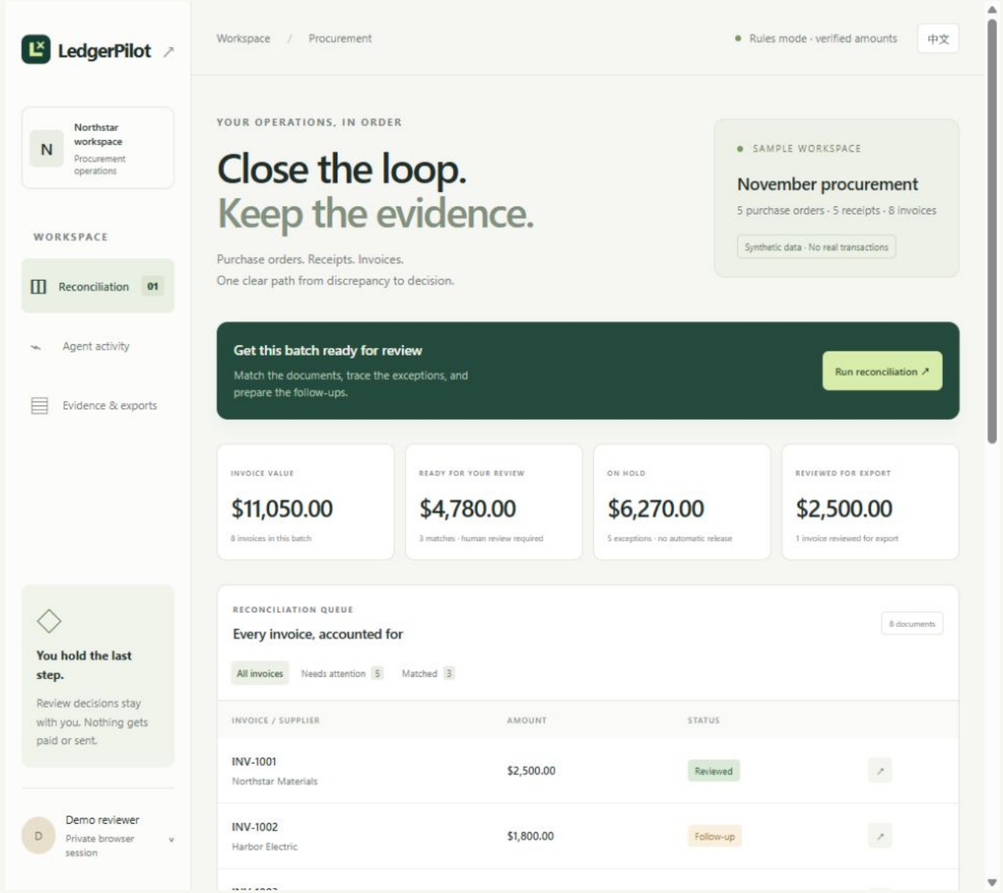
**100 units billed**

## The exception is inspectable.

The same SKU is billed for more units than the receipt supports. LedgerPilot keeps the quantities, agreed price and receipt reference together for review.

# One review queue, with a clear next action.

Actual local prototype. All organizations and records are synthetic.




Validate structured purchase orders, receipts and invoices.

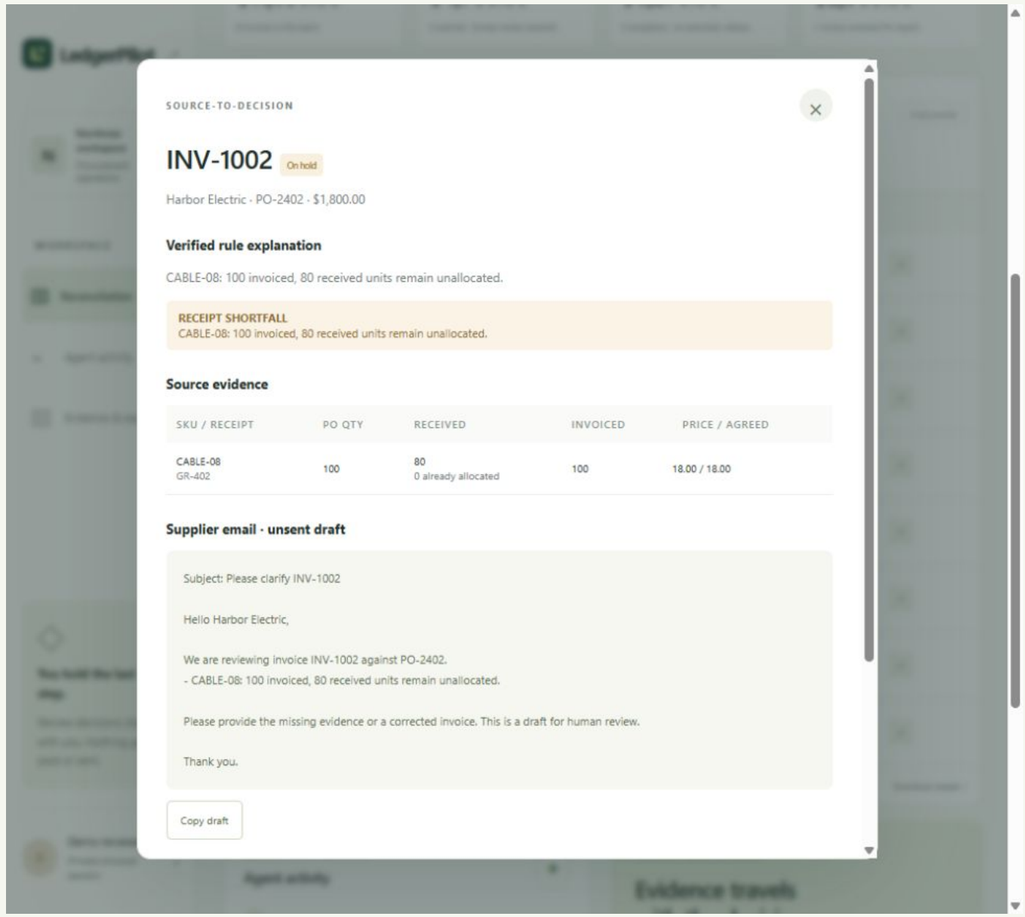
Match supplier identity, prices and available quantities.

Explain the verified exception and stage an unsent follow-up.

Review before exporting any clean invoice.

# Show the evidence behind the hold.

INV-1002 stays on hold: 100 units invoiced, 80 received under GR-402.



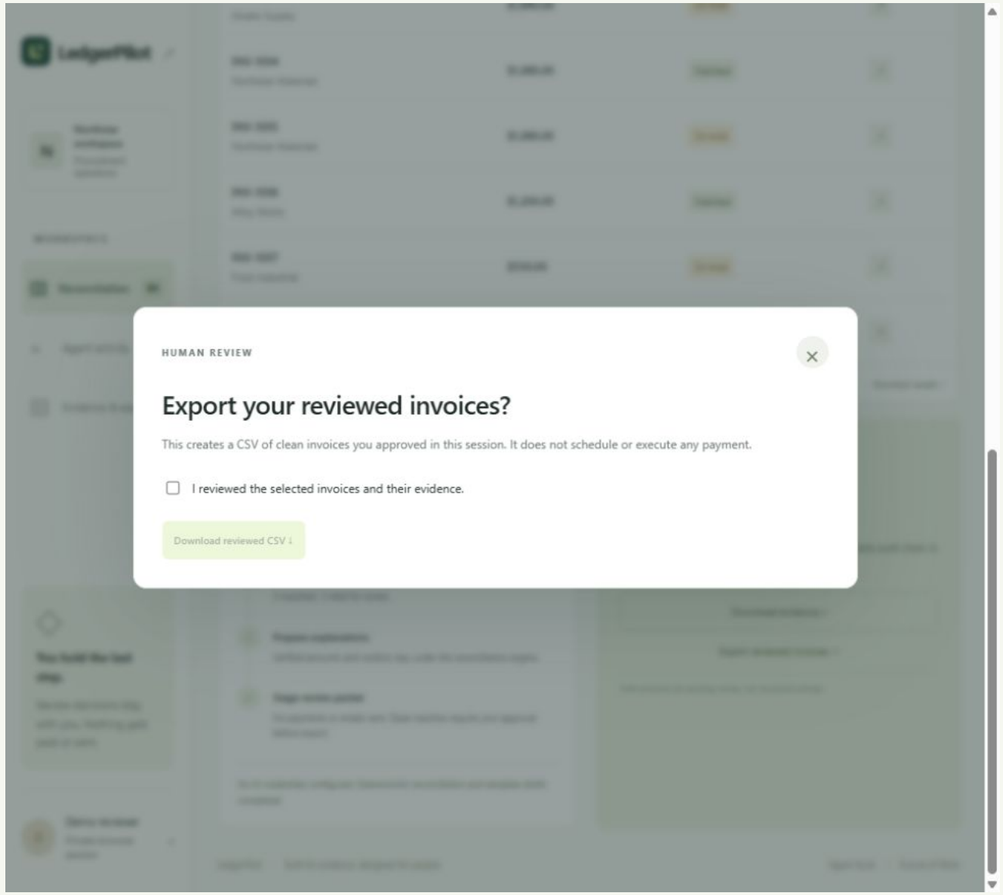

The source table exposes the document reference, quantities and agreed price.

The supplier clarification is an unsent draft. Marking follow-up sends nothing.

Held invoices cannot be approved for export, including through direct API requests.

# Approval and export are separate decisions.

Actual local confirmation UI; CSV contents verified through HTTP and container tests.




First: inspect the source evidence and approve a clean invoice.

Then: explicitly confirm review before downloading the CSV.

The tested export contains approved INV-1001 and excludes held INV-1002.

No payment or email adapter exists.

# Vultr powers the intended workflow backend.

Architecture is prepared; this is not evidence of a completed cloud deployment.

### Browser

Review queue
Source evidence
Confirmed exports

→

### Vultr VM

Node HTTP backend
Matching engine
Persistent review state

→

### Optional inference

Vultr model API
Explanations and drafts
No action tools

## The deterministic engine controls amounts and verdicts.

Caddy provides HTTPS in the planned VM deployment. The model may supply advisory text; it cannot change the matching result or approve an invoice.

# What is already verified?

Evidence is separated from the work still waiting for cloud access.

| | |
|---|---|
| **Accounting and allocation** | Independent sample totals; cumulative receipt allocation |
| **Review boundaries** | Held approvals blocked; export consent enforced |
| **State and AI boundaries** | Session isolation and restart restore; atomic AI text validation |
| **Production container** | Docker build, HTTP workflow and Compose config passed |

Six automated tests passed; VM installer syntax was checked in GitHub Actions.

Open the verification run

# A focused business hypothesis.

Intended users: small procurement teams working from structured exports.

### Existing workflow

Spreadsheet comparisons require a reviewer to keep source references and follow-ups together.

### Product proposition

One queue combines receipt allocation, visible source evidence, drafts and an export gate.

### Pilot measurement

Compare manual and assisted review on identical batches: time, false-positive holds and evidence usefulness.

Pricing concept: workspace subscription with batch limits. Customer demand, willingness to pay and time savings remain unvalidated. No market-size or revenue figures are claimed.

# From a local prototype to a reviewed entry.

Public source is available. Final hosting and submitted status remain pending.

## Prepared now

Functional rules-mode application and public GitHub repository.

Docker and HTTPS configuration; guarded Ubuntu installer.

Form text, actual UI evidence, pitch and recording script.

## Before final submission

Confirm organizer approval and event eligibility.

Confirm hosting funds; deploy and verify a real Vultr URL.

Record the hosted app; update claims; submit on lablab.ai.

Preparation and code creation began 2 November 2026, before the advertised build window. Disclose this history and confirm event-specific eligibility. The current local video is a captioned screenshot walkthrough, not a hosted-app screencast.

https://github.com/gfdy22888-design/ledgerpilot-agent-rush